1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.
   Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define MAX_LEN  256

int main() {
    char input[MAX_LEN];
    char *args[MAX_ARGS];

    printf("Parent process started. PID = %d\n", getpid());
    printf("Enter a Linux command (e.g. ls -l): ");
    fflush(stdout);

    if (fgets(input, MAX_LEN, stdin) == NULL) {
        fprintf(stderr, "Error reading input.\n");
        exit(EXIT_FAILURE);
    }
    input[strcspn(input, "\n")] = '\0';

    int i = 0;
    char *token = strtok(input, " ");
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;

    if (args[0] == NULL) {
        fprintf(stderr, "No command entered.\n");
        exit(EXIT_FAILURE);
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("Child process created. PID = %d, Parent PID = %d\n", getpid(), getppid());
        fflush(stdout);
        execvp(args[0], args);
        perror("exec failed");
        exit(EXIT_FAILURE);
    }
    else {
        int status;
        pid_t child_pid = wait(&status);
        if (WIFEXITED(status)) {
            printf("Parent (PID = %d) resumed. Child (PID = %d) exited with status %d\n",
                   getpid(), child_pid, WEXITSTATUS(status));
        } else {
            printf("Parent (PID = %d): child (PID = %d) terminated abnormally\n", getpid(), child_pid);
        }
    }
    return 0;
}
```

```
prajwal@PRAJWAL:~/os$ nano q1.c
prajwal@PRAJWAL:~/os$ gcc q1.c -o q1
prajwal@PRAJWAL:~/os$ ./q1
Parent process started. PID = 601
Enter a Linux command (e.g. ls -l): ls -1
Child process created. PID = 613, Parent PID = 601
fork
q1
q1.c
Parent (PID = 601) resumed. Child (PID = 613) exited with status 0
prajwal@PRAJWAL:~/os$
```

2.Develop a C program that uses the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the strace utility to trace all system calls generated by the command cat sample.txt. Analyze the sequence of system calls and identify the kernel services involved.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define BUF_SIZE 4096

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source_file> <destination_file>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int src_fd, dst_fd;
    ssize_t bytes_read, bytes_written;
    char buffer[BUF_SIZE];

    src_fd = open(argv[1], O_RDONLY);
    if (src_fd < 0) {
        perror("Error opening source file");
        exit(EXIT_FAILURE);
    }

    dst_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst_fd < 0) {
        perror("Error opening destination file");
        close(src_fd);
        exit(EXIT_FAILURE);
    }

    while ((bytes_read = read(src_fd, buffer, BUF_SIZE)) > 0) {
        bytes_written = write(dst_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("Write error");
            close(src_fd);
            close(dst_fd);
            exit(EXIT_FAILURE);
        }
    }

    if (bytes_read < 0) {
        perror("Read error");
    } else {
        printf("File copied successfully from %s to %s\n", argv[1], argv[2]);
    }

    close(src_fd);
    close(dst_fd);
    return 0;
}
```

```
prajwal@PRAJWAL:~/os$ nano q1.c
prajwal@PRAJWAL:~/os$ nano q2.c
prajwal@PRAJWAL:~/os$ gcc q2.c -o q2
prajwal@PRAJWAL:~/os$ ./q2
Usage: ./q2 <source_file> <destination_file>
prajwal@PRAJWAL:~/os$
```

3.Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("[Before fork] PID = %d, PPID = %d  (state: Running)\n", getpid(), getppid());
    fflush(stdout);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0) {
        printf("[Child]  PID = %d, PPID = %d  (state: Running)\n", getpid(), getppid());
        printf("[Child]  Sleeping for 3 seconds -> state becomes 'Sleeping/Waiting'.\n");
        fflush(stdout);
        sleep(3);
        printf("[Child]  Woke up, finishing -> about to Terminate\n");
        fflush(stdout);
        exit(42);
    }
    else {
        printf("[Parent] PID = %d, Child PID = %d  (state: Running)\n", getpid(), pid);
        printf("[Parent] Calling wait() -> Parent state becomes 'Sleeping/Waiting' until child terminates\n");
        fflush(stdout);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
            printf("[Parent] Child (PID %d) Terminated with exit code %d. Parent resumes 'Running'.\n",
                   pid, WEXITSTATUS(status));
    }
    return 0;
}
```

```
prajwal@PRAJWAL:~/os$ nano q2.c
prajwal@PRAJWAL:~/os$ nano q3.c
prajwal@PRAJWAL:~/os$ gcc q3.c -o q3
prajwal@PRAJWAL:~/os$ ./q3
[Before fork] PID = 706, PPID = 337  (state: Running)
[Parent] PID = 706, Child PID = 707  (state: Running)
[Parent] Calling wait() -> Parent state becomes 'Sleeping/Waiting' until child terminates
[Child]  PID = 707, PPID = 706  (state: Running)
[Child]  Sleeping for 3 seconds -> state becomes 'Sleeping/Waiting'.
[Child]  Woke up, finishing -> about to Terminate
[Parent] Child (PID 707) Terminated with exit code 42. Parent resumes 'Running'.
prajwal@PRAJWAL:~/os$
```